

Experiment No. 3

Implementation and Analysis of Spatial Filtering Techniques using Low-Pass and High-Pass Filters

Aim: To implement and analyze spatial domain filtering techniques using low-pass and high-pass filters for image smoothing, noise reduction, edge enhancement, and feature preservation using Python and OpenCV.

Purpose

Spatial filtering is an important technique in Digital Image Processing used to modify pixel values based on neighboring pixels.

- **Low-pass filters:** image smoothing and noise reduction.
- **High-pass filters:** edge detection, fine-detail enhancement, and feature extraction.

Prerequisites

- Basic Digital Image Processing
- Python programming
- OpenCV, NumPy, Matplotlib
- Image kernels and convolution
- Google Colab

Theory

Spatial filtering modifies a pixel using the values of its neighboring pixels. A small matrix called a **kernel/mask** is moved over the image.

$$[g(x,y)=\sum_i\sum_j w(i,j)f(x-i,y-j)]$$

Where:

- $f(x,y)$ = input image
- $g(x,y)$ = filtered image
- $w(i,j)$ = filter kernel

Low-Pass Filters

Reduce high-frequency information such as noise and fine details.

Examples: **Average, Gaussian, Median**

High-Pass Filters

Emphasize rapid intensity changes such as edges and fine details.

Examples: **Laplacian, Sobel**

Step 1: Import Libraries and Upload Image

Run this cell and select any image from your computer. The notebook automatically uses the uploaded image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from IPython.display import display

uploaded = files.upload()

filename = next(iter(uploaded))
image = cv2.imread(filename)

if image is None:
    raise ValueError("Image could not be loaded.")

print("Image loaded successfully:", filename)
print("Image shape:", image.shape)
```

<IPython.core.display.HTML object>

Saving 9011064.webp to 9011064.webp
Image loaded successfully: 9011064.webp
Image shape: (2160, 3840, 3)

Step 2: Display Original and Grayscale Image

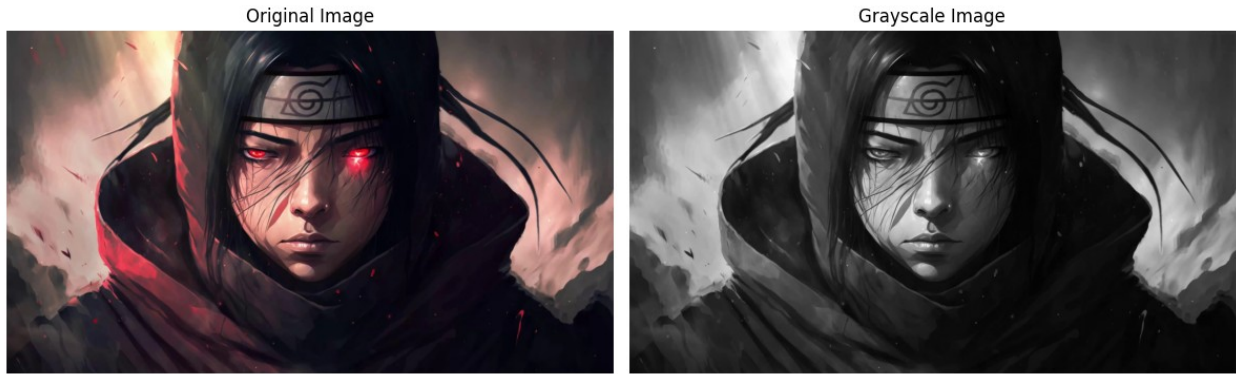
```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Step 3: Apply Low-Pass Filters

Gaussian Filter

Reduces noise and smooths the image using Gaussian-weighted neighboring pixels.

Median Filter

Replaces each pixel with the median of its neighborhood and is especially effective for salt-and-pepper noise.

Average Filter

Replaces each pixel with the average of neighboring pixels and provides basic smoothing.

```
gaussian = cv2.GaussianBlur(gray, (5, 5), 0)
median = cv2.medianBlur(gray, 5)
average = cv2.blur(gray, (5, 5))

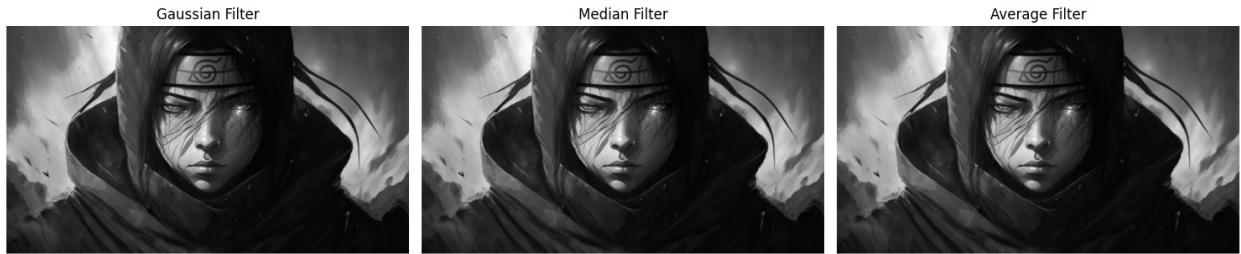
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(gaussian, cmap="gray")
plt.title("Gaussian Filter")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(median, cmap="gray")
plt.title("Median Filter")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(average, cmap="gray")
plt.title("Average Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Step 4: Apply High-Pass Filters

Laplacian Filter

Highlights rapid intensity changes, edges, and fine details.

Sobel Filter

Calculates image gradients. Sobel X mainly detects vertical edges, while Sobel Y mainly detects horizontal edges.

```
laplacian = cv2.Laplacian(gray, cv2.CV_64F)
laplacian = cv2.convertScaleAbs(laplacian)

sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_x = cv2.convertScaleAbs(sobel_x)

sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel_y = cv2.convertScaleAbs(sobel_y)

sobel_combined = cv2.addWeighted(sobel_x, 0.5, sobel_y, 0.5, 0)

plt.figure(figsize=(15, 7))

plt.subplot(2, 2, 1)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Filter")
plt.axis("off")

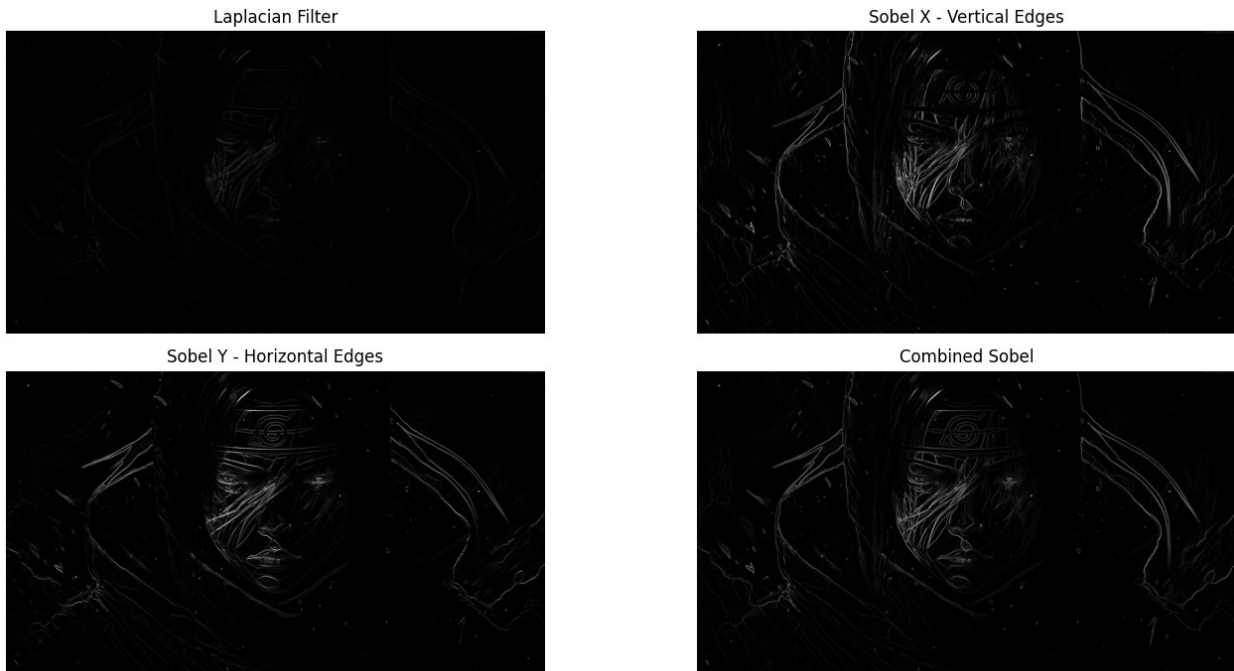
plt.subplot(2, 2, 2)
plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X - Vertical Edges")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(sobel_y, cmap="gray")
plt.title("Sobel Y - Horizontal Edges")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(sobel_combined, cmap="gray")
```

```
plt.title("Combined Sobel")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Step 5: Complete Comparison of All Filters

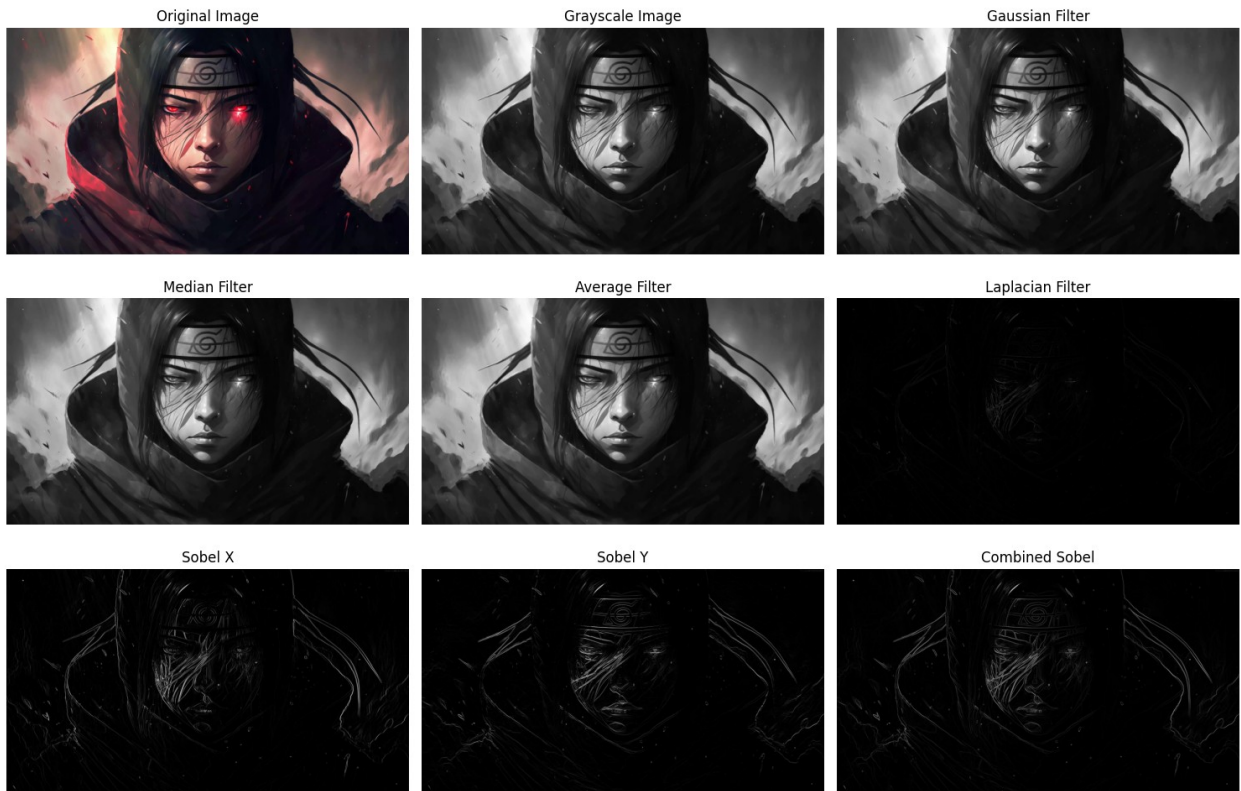
```
plt.figure(figsize=(15, 10))

results = [
    (image_rgb, "Original Image", None),
    (gray, "Grayscale Image", "gray"),
    (gaussian, "Gaussian Filter", "gray"),
    (median, "Median Filter", "gray"),
    (average, "Average Filter", "gray"),
    (laplacian, "Laplacian Filter", "gray"),
    (sobel_x, "Sobel X", "gray"),
    (sobel_y, "Sobel Y", "gray"),
    (sobel_combined, "Combined Sobel", "gray")
]

for i, (img, title, cmap) in enumerate(results, 1):
    plt.subplot(3, 3, i)
    if cmap:
        plt.imshow(img, cmap=cmap)
    else:
        plt.imshow(img)
```

```
plt.title(title)
plt.axis("off")

plt.tight_layout()
plt.show()
```



Observations

- **Gaussian Filter:** Smooths the image and reduces random noise while retaining important structures reasonably well.
- **Median Filter:** Very effective for salt-and-pepper noise and generally preserves edges better than average filtering.
- **Average Filter:** Provides simple smoothing but may blur edges and fine details.
- **Laplacian Filter:** Highlights edges and fine details but can be sensitive to noise.
- **Sobel X:** Mainly detects vertical edges.
- **Sobel Y:** Mainly detects horizontal edges.
- **Combined Sobel:** Represents edges detected in both directions.

Filter Comparison

Filter	Type	Main Purpose	Edge Preservation
Average	Low-pass	Smoothing	Low
Gaussian	Low-pass	Noise reduction and smoothing	Better than Average

Filter	Type	Main Purpose	Edge Preservation
Median	Non-linear	Salt-and-pepper noise removal	Very good
Laplacian	High-pass	Edge enhancement	High
Sobel X	High-pass	Vertical edge detection	High
Sobel Y	High-pass	Horizontal edge detection	High

Questions and Answers

Q1. What is spatial filtering? How is it used in digital image processing?

Spatial filtering modifies the intensity of a pixel using neighboring pixels through a kernel or mask. It is used for noise removal, smoothing, sharpening, edge detection, and feature enhancement.

Q2. Differentiate between Low-Pass Filters and High-Pass Filters with suitable examples.

Low-pass filters reduce high-frequency components and are used for smoothing and noise reduction. Examples are Gaussian, Median, and Average filters. High-pass filters emphasize high-frequency components and are used for edge detection and detail enhancement. Examples are Sobel and Laplacian filters.

Q3. Compare Average Filter, Gaussian Filter, and Median Filter.

- **Average:** Uses the arithmetic mean of neighboring pixels; basic smoothing.
- **Gaussian:** Uses Gaussian-weighted neighboring pixels; smooth noise reduction.
- **Median:** Uses the median neighborhood value; excellent for salt-and-pepper noise and edge preservation.

Q4. Why is the Median Filter effective for salt-and-pepper noise?

Salt-and-pepper noise contains isolated extreme black or white pixels. The median operation rejects these extreme values and selects a representative middle value, removing noise while preserving edges.

Q5. Explain the role of convolution kernels in spatial filtering.

A convolution kernel is a small matrix that moves across an image and combines neighboring pixel values according to its coefficients. Different kernels perform smoothing, sharpening, or edge detection.

Q6. What is the purpose of Sobel and Laplacian operators?

Sobel calculates first-order image gradients in horizontal and vertical directions. Laplacian calculates the second-order derivative and highlights rapid intensity changes, edges, and fine details.

Q7. Why are filtering operations essential preprocessing steps in computer vision?

Filtering reduces noise, improves image quality, enhances useful structures, and makes later operations such as segmentation, feature extraction, and object detection more reliable.

Q8. Discuss the trade-off between image smoothing and edge preservation.

Strong smoothing reduces noise but can blur important edges and fine details. Weak smoothing preserves details but leaves more noise. Therefore, filter type and kernel size should be selected carefully.

Q9. Mention four real-world applications of spatial filtering.

1. Medical imaging
2. Surveillance systems
3. Autonomous vehicles
4. Remote sensing

Q10. Compare spatial domain and frequency domain filtering.

Spatial filtering operates directly on image pixels using kernels and is convenient for local operations such as smoothing and edge detection. Frequency-domain filtering transforms an image into frequency components, filters those components, and then applies an inverse transform. It is useful when controlling specific frequency characteristics.